Collision de Hash

Projet Développement Logiciels Cryptographiques

Théo Klingler, Quentin Lombardo, Abir Aroui

2024-02-02

Résumé

Le projet avait pour objectif d'utiliser les algorithmes de Floyd et de Brent afin de réaliser une collision partielle de hash. Chacune des parties présenté dans le rapport va présenté les étapes qui nous ont permis de réaliser une collision partielle de hash entre deux images au format bitmap.

Étape 1

La première étape consiste à faire une simple programme nommé `md5_small` affichant les *t* bits d'un hash MD5 donné par un fichier en entrée. Le programme étant plutôt simple, nous ne rentrerons pas dans les détails ici. Voici le code qui réalise cet étape:

```c
int main(int argc, char *argv[])
{
    unsigned int hash_size, hash_bytes;
    unsigned char *file = NULL;
    if(argc != 3) {
        fprintf(stderr, "Usage: %s <fichier> <taille>\n", argv[0]);
        exit(1);
    }

    hash_size = (unsigned int)atoi(argv[2]);
    hash_bytes = ((hash_size - 1) / 8) + 1;

    long fsize = file_get_contents(argv[1], &file);
    unsigned char *hash = calloc(hash_bytes, sizeof(unsigned char));

    if(md5_small(hash, hash_bytes, hash_size, file, fsize) != 0) {
        exit(1);
    }

    printh(hash, hash_bytes);
    printf("\n");

    free(file);
    free(hash);
    return 0;
}
```

Étape 2

L'étape 2 a pour but de réaliser l'algorithme de Floyd. Cet algorithme permet de trouver une boucle dans une liste chainée en utilisant deux pointeurs se mouvant dans la liste à une vitesse différente. On

va nommer “Lièvre” le pointeur rapide et “Tortue” le pointeur lent. L’algorithme se découpe en deux phases distinctes. ## Phase 1

Au départ de l’algorithme, les deux pointeurs se trouvent au même endroit. Puis à chaque itération, on fait avancer Tortue de 1 index quand Lièvre va avancer de 2 indexes. En itérant selon ces contraintes, et selon la longueur du cycle, Lièvre et Tortue vont finir par ce rencontrer. Voici l’explication précédente sous forme de schéma:

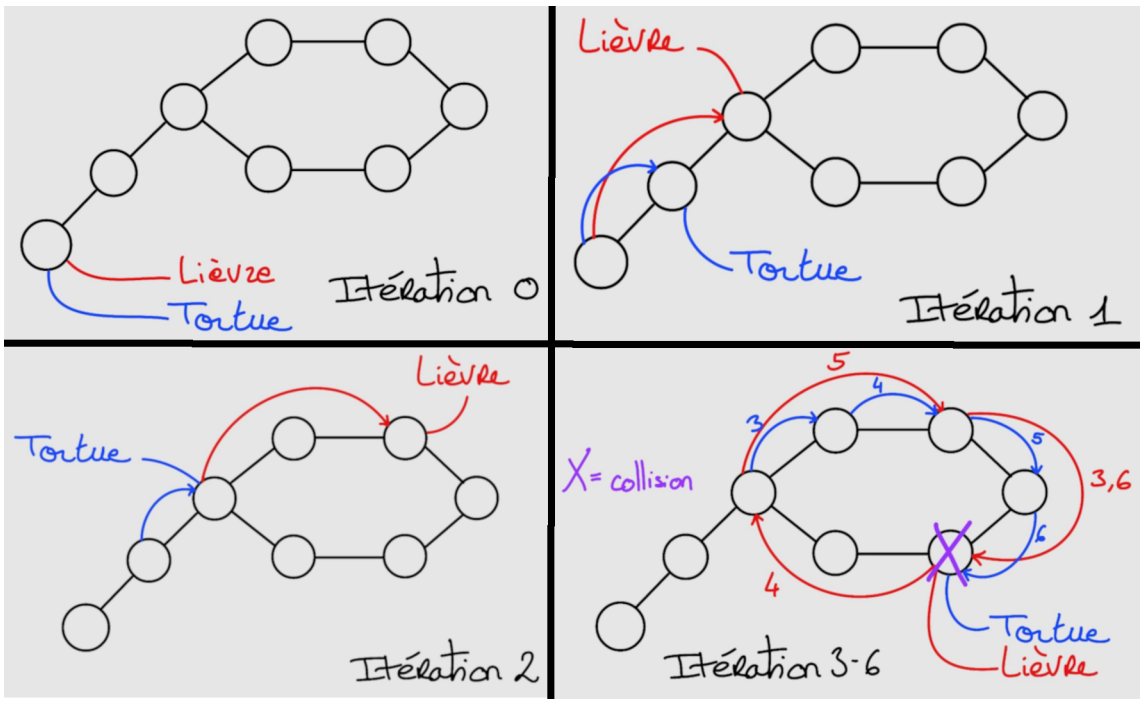

Figure 1: Floyd phase 1

Dans le cadre de notre projet, la liste chaînée est représentée par un enchaînement successif de fonctions de hash sur un contenu donné. La vitesse de Tortue est d’un hash par itération quand celle de Lièvre est de deux. Nous avons traduit cette phase 1 de la façon suivante dans le code:

```
collision floyd(size_t (*f)(size_t, byte *, byte **), size_t taille,
    byte *seed){
    // Création des variables
    unsigned int b_taille = ((taille - 1) / 8) + 1;
    byte *lievre, *tortue;
    size_t s_lievre, s_tortue;
    collision c = {NULL, NULL, 0};

    // Initialisation des variables
    byte *h_lievre = (byte*) calloc(b_taille, sizeof(byte));
    byte *h_tortue = (byte*) calloc(b_taille, sizeof(byte));
    for (int i = 0; i < b_taille; ++i)
```

```
        h_tortue[i] = h_lievre[i] = seed[i];

    // Boucle de la phase 1
    do {
        s_tortue = f(b_taille, h_tortue, &tortue);
        md5_small(h_tortue, b_taille, taille, tortue, s_tortue);

        s_lievre = f(b_taille, h_lievre, &lievre);
        md5_small(h_lievre, b_taille, taille, lievre, s_lievre);
        s_lievre = f(b_taille, h_lievre, &lievre);
        md5_small(h_lievre, b_taille, taille, lievre, s_lievre);
        c.iter += 3;
    } while(memcmp(h_tortue, h_lievre, b_taille));
    // [...]
```

> Le code ci-dessus (et les prochains) ont été légèrement modifiés pour s'adapter à l'étape du rapport. Le code source livrée est donc légèrement different pour répondre au futurs étapes du processus de collision.

Au départ de la fonction, on initialises nos variables:

- L'utilisateur précise la taille en bits. La valeur *b_taille* permet donc de changer cette valeur en octets pour effectuer nos allocations mémoires.
- Les pointeurs Lièvre et Tortue sont représentés comme il suit: `lievre` et `tortue` sont les données que l'on va hasher. `h_lievre` et `h_tortue` sont les hash générés par ces données. Dans le cas de Floyd_1, `lievre` et `tortue` vont contenir des hashs, mais dans la suite du rapport, nous les introduiront en tant qu'images.
- La structure collision représente les informations qui vont être retournées à l'utilisateur, les deux premières valeurs sont les antécédents ayant une collision partielle sur le hash et la dernière valeur représente le nombre de fois ou la fonction MD5 est appelé dans le programme.

Ensuite, on alloue la mémoire nécessaire et on initialise les pointeurs `h_lievre` et `h_tortue` sur la valeur de départ. Cette valeur est donnée par une seed récupéré dans le fichier `/dev/urandom`.

Enfin, on applique les itérations dans une boucle *while* qui compare la valeur des deux hashs. Les itérations sont appliquées sur les pointeurs avec la fonction *f* et *md5_small*:

- *f* est une simple fonction d'identité qui place la valeur de `h_tortue` dans `tortue`.
- *md5_small* hash le contenu de `tortue` puis place la valeur obtenue dans `h_tortue`, seul les *b* bits du hash MD5 sont gardés.
- La même opération est réalisé deux fois pour `h_lievre` et `lievre`.
- Quand `h_lievre` et `h_tortue` ont la même valeur, la boucle s'arrête et la phase 1 se termine.

Phase 2

La conclusion de la phase 1 nous permet de retrouver la longueur du cycle et d'avoir notre point de départ pour la phase 2. Cependant nous n'avons toujours pas trouvé la collision partielle recherché. Le but de la phase 2 est de trouver deux antécédents différents pointant sur une valeur avec *b* bits identiques.

Au lancement de la phase 2, le pointeur Tortue est replacé à la valeur de la seed quand le pointeur Lièvre reste à l'endroit de la collision de la première phase. La vitesse de Lièvre est aussi modifiée pour se réduire à 1 par itération au lieu de 2 précédemment. Le déroulement de la phase va alors pouvoir s'effectuer avec les deux pointeurs avançant à la même vitesse par itération. De cette manière, la prochaine collision entre les deux pointeurs va provenir d'antécédents différents mais retournant la même valeur comme le montre le schéma ci-dessous:

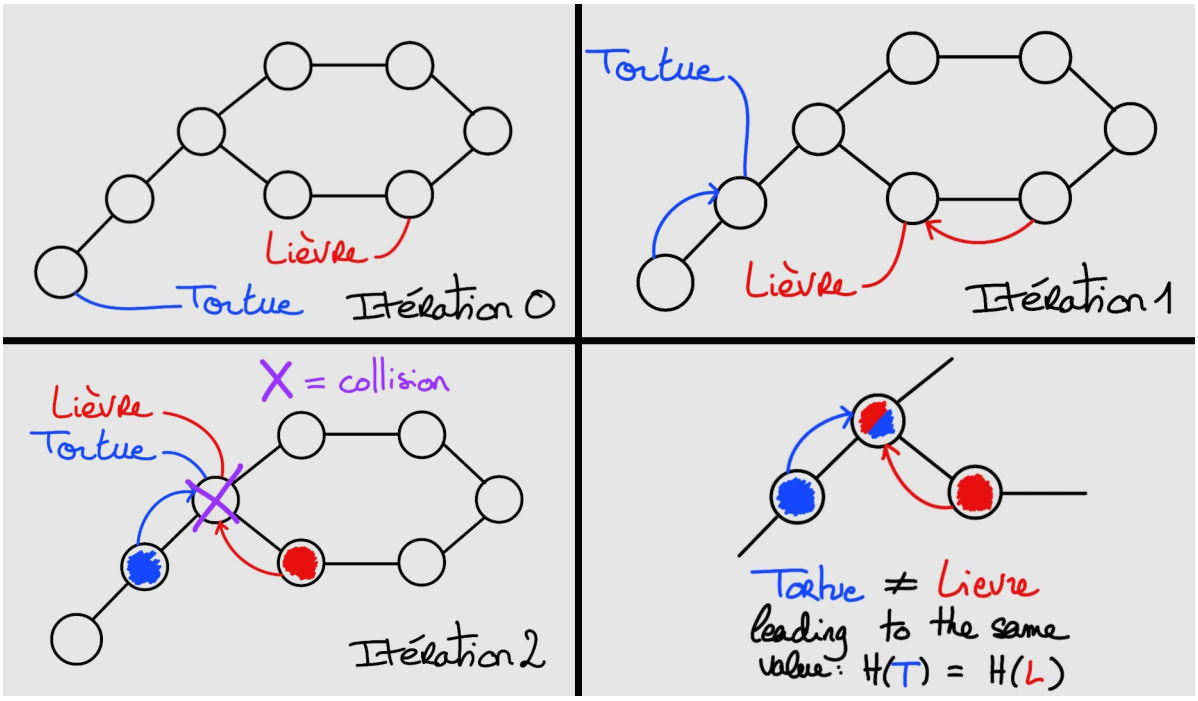

Figure 2: Floyd phase 2

Cette phase a été traduite dans le code comme il suit:

```
    // [...]
    // On replace le pointeur de Tortue
    for (int i = 0; i < b_taille; ++i)
        h_tortue[i] = seed[i];

    // Boucle de la phase 2
```

```
    do {
        s_tortue = f(b_taille, h_tortue, &tortue);
        md5_small(h_tortue, b_taille, taille, tortue, s_tortue);

        s_lievre = f(b_taille, h_lievre, &lievre);
        md5_small(h_lievre, b_taille, taille, lievre, s_lievre);
        c.iter += 2;
    } while(memcmp(h_tortue, h_lievre, b_taille));
    c.lievre = lievre;
    c.tortue = tortue;
    return c;
}
```

La phase 2 débute donc avec la réinitialisation de la valeur de `h_tortue` par la seed. Puis c'est simplement la même boucle while que précédemment qui s'effectue. Lièvre est désormais itéré qu'une seule fois ce qui traduit bien la phase 2. Lorsque les deux pointeurs valent la même valeur, on sort de la boucle *while* et le résultat contenant les antécédents de la collision sont retournées à l'utilisateur à travers notre structure *c*.

Déroulement et Analyse

Voici un premier exemple de déroulement de notre algorithme de Floyd. Dans l'exemple, nous avons choisi *t=32*. Voici les résultats que nous avons obtenus:

```
35e0e748 20e2c5cd
f33d6948 64581731
006fcd8b af0ad9f9
1426a090 31a66b20
14c1bccb 14c1bccb
1426a090
31a66b20
Itérations: 229558
./floyd_1 32  0.52s user 0.18s system 93% cpu 0.746 total
babbz:collisions-de-hachage/ (5d443c0*) $ echo -ne "1426a090" | xxd -r -p | md5sum
1c9e438be3c696f041c9d19f14c1bccb  -
babbz:collisions-de-hachage/ (5d443c0*) $ echo -ne "31a66b20" | xxd -r -p | md5sum
8bf56ee28fe4c70890d751d814c1bccb  -
```

Figure 3: Déroulement Floyd exemple 1

Comme le montre les résultats, notre collision s'effectue bien sur les 4 octets indiquées (collision sur `14c1bccb`). Ce résultat à été obtenu au bout de *229558* itérations de MD5 soit environ `2**18` et en à peine une demi seconde.

Voici un deuxième exemple sur un *t* plus grand: *t=50*.

```
0173e8d3cd822f 0303e58a253252
0322d24bb8040a 015010c4c6ca29
02fd08996e2199 0231769fde0f00
01412dedad2475 021b14d98d4571
002b75f7d89714 002b75f7d89714
01412dedad24
021b14d98d45
Itérations: 112874699
./floyd_1 50  256.92s user 81.68s system 95% cpu 5:53.91 total
babbz:collisions-de-hachage/ (5d443c0*) $ echo -ne "01412dedad2475" | xxd -r -p | md5sum
c9262620250c1bdb051c2b75f7d89714  -
babbz:collisions-de-hachage/ (5d443c0*) $ echo -ne "021b14d98d4571" | xxd -r -p | md5sum
585c4495cda645a96ce02b75f7d89714  -
```

Figure 4: Déroulement Floyd exemple 2

Cette fois ci, la collision s'est obtenu au bout de *112874699* itérations de MD5 soit environ `2**27`. Ce qui est drastiquement plus que précédemment même si le nombre d'iterations peux grandement varier en fonction de la seed. Cette fois ci la collision s'effectue sur `02b75f7d89714` qui correspond à *50* bits.

De plus, on sait que le nombre d'itération de MD5 théorique est censé évoluer selon la formule suivante:

$$O(n) = 2^{\frac{n}{2}}$$

Et on remarque que nos résultats suivent de manière très similaire la formule précédente:

$$\frac{0.52}{2^{\frac{32}{2}}} * 2^{\frac{50}{2}} = 266.24$$

Ce qui représente un écart d'environ *-3%* entre notre valeur et la valeur estimée:

$$\frac{256.92 - 266.24}{256.92} \approx -3\%$$

On peut effectuer le même calcul avec le nombre d'itérations, ce qui nous donne:

$$\frac{229558}{2^{\frac{32}{2}}} * 2^{\frac{50}{2}} = 117533696$$

Cette fois cela représente environ *4%* de déviation entre notre valeur et la valeur théorique:

$$\frac{112874669 - 117533696}{112874669} \approx -4\%$$

Nos résultats étant très proche de la théorie mathématique, on peut affirmer le bon déroulement et développement de notre programme. Afin de compléter l'analyse sur Floyd_1, voici un tableau représentant un benchmark selon le nombre de bits (obtenu à l'aide de l'outil hyperfine):

# bits	Mean [ms]	Min [ms]	Max [ms]
16	4.8 ± 0.4	4.4	5.4
18	11.7 ± 5.5	6.9	21.0
20	10.0 ± 4.6	7.1	18.0
22	11.1 ± 5.3	8.6	20.6
24	7.8 ± 3.1	5.0	12.6
26	36.7 ± 16.2	20.9	54.8
28	123.6 ± 70.1	61.7	203.4
30	191.7 ± 132.3	87.6	399.3
32	360.0 ± 139.9	168.5	515.1
34	679.2 ± 424.6	216.4	1136.3
36	1936.4 ± 1555.2	649.1	4197.5
38	2141.6 ± 2499.0	312.2	6176.8
40	4432.1 ± 1826.1	2999.2	7364.2
42	15842.6 ± 6946.6	10311.7	27629.1
44	29022.4 ± 12473.6	19117.9	43307.2
46	27346.2 ± 13427.8	9125.3	43133.6
48	146670.6 ± 38094.7	81032.9	177367.7
50	220177.0 ± 17216.0	194825.7	239863.2

Ce tableau nous prouve une fois de plus le bien fondé de notre programme grace au temps d'executions qui respectent assez clairement l'évolution selon la formule théorique.

Étape 3

Point de départ

Le programme actuel permet d'effectuer une collision partielles entre deux valeurs aléatoires. Cependant il est difficile d'imaginer un quelconque intérêt à celui-ci. Une collision de hash peut se trouver intéressante pour une personne malveillante dans le cadre d'une signature numérique de document.

L'attaquant va alors remplacer un document signé (représenté par un hash) par un autre document valant le même hash mais ayant un contenu modifié pour lui être profitable. Ce cadre nécessite donc de mettre au point un moyen pour traiter des hash d'images plutôt que des hash de hashs.

Nous avons donc mis en place une fonction `hash_to_bitmap` qui prend en entrée une image et un hash. Elle va utiliser le hash pour modifier l'image de manière indistinguable par un œil humain de la façon suivante:

```
size_t hash_to_bitmap(size_t taille, hash *hash, h_file image, h_file *
    out) {
    int i;
    copy_hfile(out, image);
    for (i = 0; i < taille * 8; i++) {
        (*out)->data[(*out)->len - i - 1] ^= (hash[i/8] >> (i%8)) & 1;
    }
    return (*out)->len;
}
```

Pour un hash de *n* bits, ce code va changer le bit de poids faible des *n* derniers octets de l'image. On va donc itérer sur chaque octet de l'image, en partant du dernier octet.

Le `i % 8` va permettre d'indexer chaque octet du hash selon le bit à XOR avec l'octet de l'image correspondant. En faisant `>> (i%8)`, on place le bit voulu en bit de poids faible du hash, ce qui nous permet d'appliquer 1 en *bitmask*. Pour un octet comme le suivant `0b01000101`, ce code va donc XOR les huit derniers octets de l'image avec `[0, 1, 0, 0, 0, 1, 0, 1]`.

Ainsi à chaque itération de l'algorithme de Floyd, le hash retourné est transformé en l'image souhaitée en sortie par l'attaquant. Au moment de la collision, on aura alors simplement à retrouver les deux images qui ont permis cette collision.

Malheureusement, l'utilisation (même adapté) de l'algorithme de Floyd présenté à l'étape 2 permet de trouver une collision qu'entre deux images similaires seulement. La seed donnée au départ de l'algorithme est unique et aucune seconde image préparée par l'attaquant n'intervient dans l'algorithme. C'est pourquoi il est nécessaire d'adapter Floyd pour répondre à cette problématique.

Floyd 3

L'algorithme Floyd 3 à été pensé pour permettre une collision partielle entre deux hachés d'images différentes. Pour cela, il nous a fallut modifier le fonctionnement de l'algorithme de Floyd. Notre modification se représente schématiquement comme il suit:

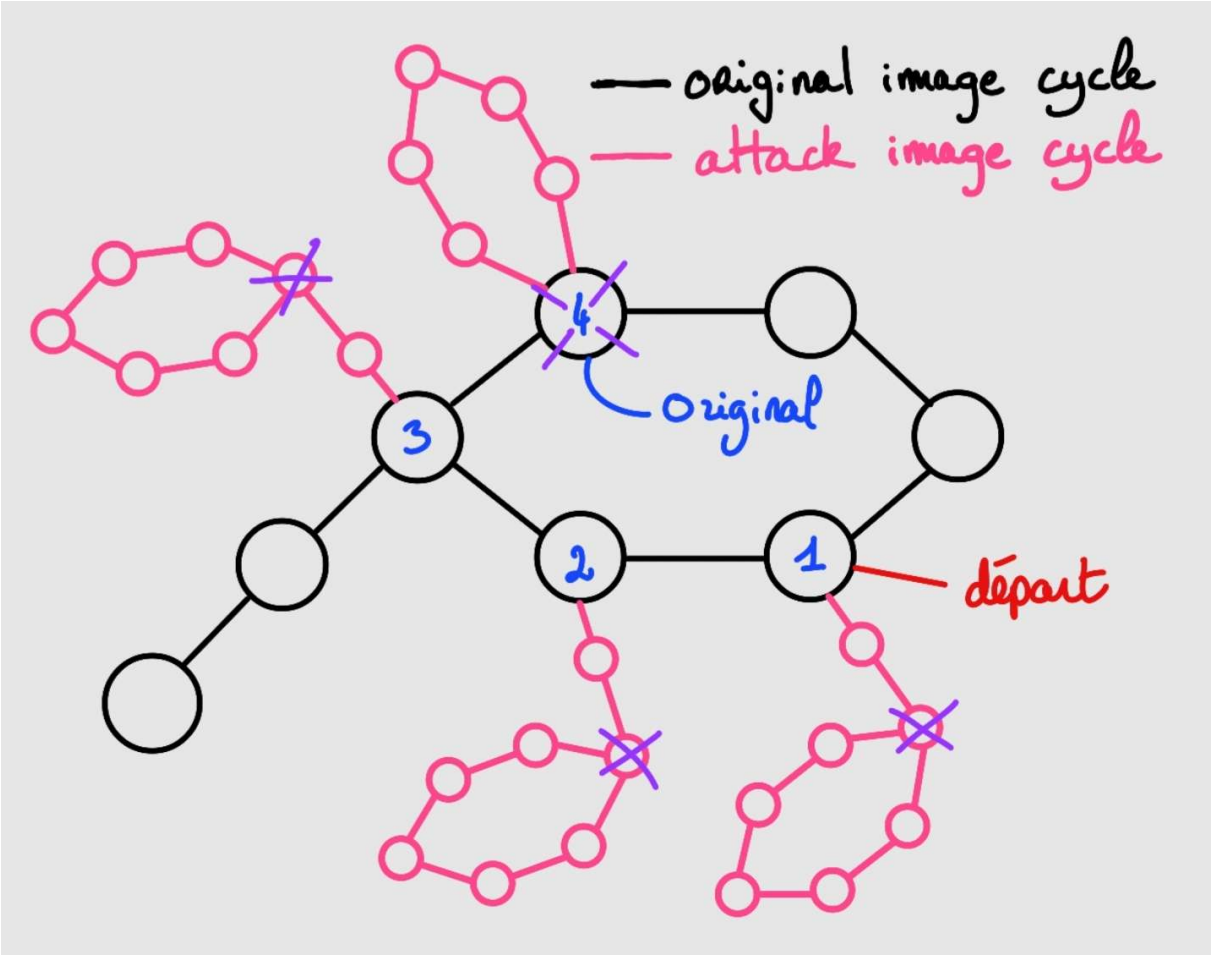

Figure 5: Floyd 3 phase image d'attaque

Comme Floyd, l'algorithme se déroule en deux parties distinctes: La première partie réutilise la phase 1 de l'algorithme de Floyd. Cette phase 1 est réalisé sur l'image dite "originale" (dont on veux une image différente qui génère un hash similaire). On utilise une seed random (que l'on va appeler *seed1*) pour générer la première image comme expliqué plus haut. Puis on lance le processus de phase 1 de Floyd. C'est lorsque la collision va s'effectuer sur l'image originale que notre méthode va s'éloigner de l'algorithme original de Floyd.

À ce moment là, on va lancer une deuxième phase 1 de Floyd avec l'image dite "d'attaque" (celle que l'on va substituer à l'originale et dont le hash sera similaire à celle-ci). Cette fois, la seed utilisé sera le hash ou la collision s'est effectuée sur l'image originale lors de la phase précédente (que l'on va appeler *seed2*). L'algorithme de Floyd en phase 1 va alors s'effectuer sur l'image d'attaque en traduisant à chaque fois le hash résultant en une nouvelle image d'attaque. À chaque itération on va effectuer une comparaison entre le hash de l'image d'attaque et la *seed2*. Durant ce processus, plusieurs résultats

sont possibles:

- La *seed2* est comprise dans le cycle de hash de l'image d'attaque. Dans ce cas, une collision va s'effectuer entre le hash de l'image d'attaque et la *seed2*. Ce qui signifie qu'une collision existe entre nos deux images, il suffit alors de les récupérer et de les retourner à l'utilisateur.
- La *seed2* n'est pas comprise dans le cycle. Dans ce cas, c'est une collision entre le Lièvre et la Tortue de la phase sur l'image d'attaque qui va s'effectuer. Celle-ci signifie que le cycle à entièrement été vérifié, et qu'aucun hash de celui-ci n'entre en collision avec la *seed2*.

Dans le deuxième cas, on va remplacer la *seed2* par un autre hash du cycle de l'image originale. Le processus de la phase 1 sur l'image d'attaque est alors effectuée avec cette nouvelle seed. On re-itère le processus jusqu'à ce qu'une collision soit trouvée ou que tout les hash présent dans le cycle de l'image originale aient été testés.

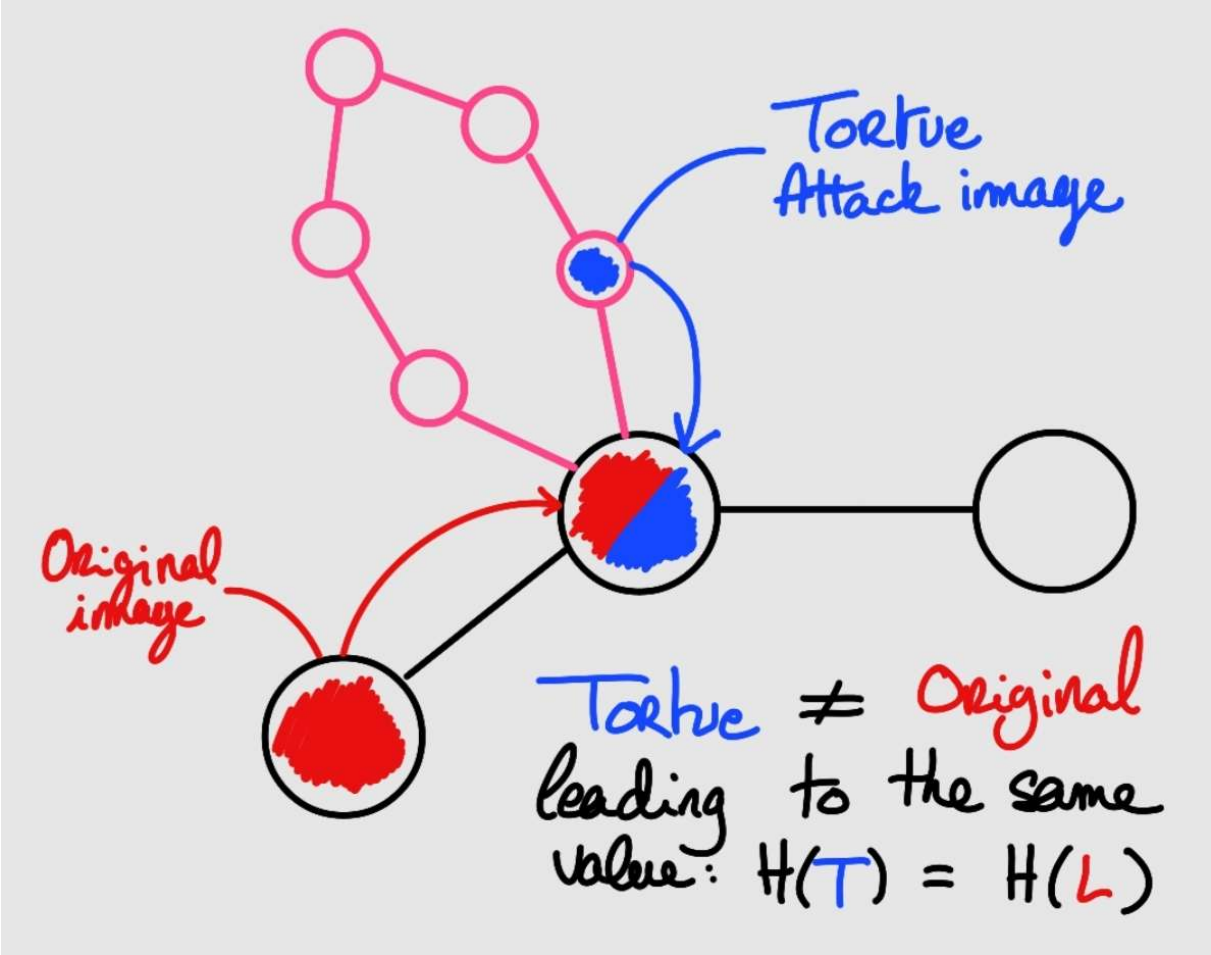

Figure 6: Floyd 3 collision d'images diffèrentes

Nous avons adapté la fonction Floyd_3 tel qu'il suit dans notre code:

```
collision floyd(size_t (*f)(size_t, byte *, h_file, h_file *), h_file entree, h_file sortie, size_t taille, byte *seed){
    // Creation des variables
    unsigned int b_taille = ((taille - 1) / 8) + 1;
    h_file lievre, tortue, original;
    size_t s_lievre, s_tortue, s_original;
    collision c = {NULL, NULL, 0};

    // Initialisation des variables
    byte *h_lievre = (byte*) calloc(b_taille, sizeof(byte));
    byte *h_tortue = (byte*) calloc(b_taille, sizeof(byte));
    byte *h_original = (byte*) calloc(b_taille, sizeof(byte));
    byte *h_depart = (byte*) calloc(b_taille, sizeof(byte));
    for (int i = 0; i < b_taille; ++i)
        h_tortue[i] = h_lievre[i] = seed[i];

    // Boucle de la phase 1 sur l'image originale
    do {
        s_tortue = f(b_taille, h_tortue, entree, &tortue);
        md5_small(h_tortue, b_taille, taille, tortue->data, s_tortue);

        s_lievre = f(b_taille, h_lievre, entree, &lievre);
        md5_small(h_lievre, b_taille, taille, lievre->data, s_lievre);
        s_lievre = f(b_taille, h_lievre, entree, &lievre);
        md5_small(h_lievre, b_taille, taille, lievre->data, s_lievre);
        c.iter += 3;
    } while(memcmp(h_tortue, h_lievre, b_taille));
    // [...]
```

Comme sur l'algorithme de Floyd classique, on crée puis initialise les variables. Cependant cette fois ci plusieurs choses s'ajoutent:

- La structure `h_file` permet de simplifier la gestion des pointeurs, c'est un *'fat pointer'* qui contient la taille de la data ainsi que la data. Elle permet de stocker les images en format bitmap dans un tableau de bytes.
- La variable 'original' représente l'image dont le hash va être utilisé en tant que *seed2* pour la phase suivante, ce hash sera stocker dans la variable `h_original`.
- La variable `h_depart` va contenir le hash de collision de la première phase. Lorsque `h_depart` et `h_original` valent là même valeur, on sait que le programme s'est terminé et qu'aucune collision n'a été trouvée avec le *seed1*.

La phase 1 de l'image originale ce termine après la boucle *while*, on a à ce moment là: `h_pierre = h_arbre = h_lievre = h_tortue`. C'est alors que commence la phase 2 de notre programme.

```c
    // [...]
    copy_hfile(&original, tortue);  // original <= tortue
    memcpy(h_depart, h_tortue, b_taille);  // h_depart = h_tortuse

    // Boucle sur le cycle de l'image originale
    do {
        s_original = f(b_taille, h_original, entree, &original);
        md5_small(h_original, b_taille, taille, original->data,
            s_original);

        for (int i = 0; i < b_taille; ++i)
            h_tortue[i] = h_lievre[i] = h_original[i];

        // Boucle sur l'image d'attaque
        do {
            s_tortue = f(b_taille, h_tortue, sortie, &tortue);
            md5_small(h_tortue, b_taille, taille, tortue->data,
                s_tortue);

            s_lievre = f(b_taille, h_lievre, sortie, &lievre);
            md5_small(h_lievre, b_taille, taille, lievre->data,
                s_lievre);
            s_lievre = f(b_taille, h_lievre, sortie, &lievre);
            md5_small(h_lievre, b_taille, taille, lievre->data,
                s_lievre);
            c.iter += 3;
        } while(memcmp(h_tortue, h_lievre, b_taille));

    } while(memcmp(h_original, h_lievre, b_taille) && memcmp(h_original
        , h_depart, b_taille));

    if(!memcmp(h_original, h_depart, b_taille)) {
        fprintf(stderr, "Mauvaise seed. Désolé.\n");
        exit(EX_SOFTWARE);
    }
    c.original = original;
    c.attaque = lievre;
    return c;
}
```

Au départ de la phase 2, on replace nos pointeurs et nos images comme il suit:

- L'image qui à provoqué la collision (`tortue`) est placée dans l'image originale.
- Le hash qui contient la collision est placée dans `h_depart` qui ne sera plus modifiée de tout le programme.

La première boucle *while* gère l'avancement du cycle de l'image originale, c'est ici qu'est comparé `h_depart` avec `h_original` pour savoir quand le programme doit s'arrêter. La boucle s'arrête aussi si la collision est trouvé dans la première boucle, c'est a dire que `h_original = h_lievre`

(ou `h_tortue`). On parcours le cycle de l'image original comme on l'a parcouru lors de la première phase.

La deuxième boucle *while* gère l'avancement du cycle de l'image d'attaque, c'est ici qu'est comparé `h_tortue` avec `h_lievre`. Une collision entre ces deux dans notre boucle signifie forcément que la totalité des hashs du cycle a été parcouru. Et pour que cet collision nous soit utile dans notre contexte, il est nécessaire que `h_lievre = h_original`. Si ce n'est pas le cas, c'est que le hash original n'est pas contenu dans notre boucle donc qu'il n'existe pas d'image d'attaque ayant une collision avec ce hash de l'image originale.

À la sortie de ces deux boucles, on vérifie que l'on est sortie grace à une collision entre nos deux images plutôt qu'a cause de l'échec de notre tentative de collision. Dans le cas ou la collision à été trouvé, on retourne nos deux images permettant celle-ci à travers notre structure *c*.

Déroulement

Voici un exemple de déroulement de notre algorithme dans son implémentation naif sur *14* bits de collision:

```
babbz:collisions-de-hachage/ (master) $ time ./floyd_3 tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15
Itérations: 10687
./floyd_3 tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15  38.33s user 0.00s system 99% cpu 38.394 total
babbz:collisions-de-hachage/ (master) $ time ./floyd_3 tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15
Mauvaise seed. Désolé.
./floyd_3 tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15  131.22s user 0.00s system 99% cpu 2:11.39 total
babbz:collisions-de-hachage/ (master) $ time ./floyd_3 tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15
Mauvaise seed. Désolé.
./floyd_3 tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15  131.69s user 0.01s system 99% cpu 2:11.90 total
babbz:collisions-de-hachage/ (master) $ time ./floyd_3 tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15
Itérations: 1942
./floyd_3 tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15  6.95s user 0.00s system 99% cpu 6.967 total
babbz:collisions-de-hachage/ (master) $ time ./floyd_3 tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15
Itérations: 2622
./floyd_3 tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15  9.45s user 0.01s system 99% cpu 9.516 total
babbz:collisions-de-hachage/ (master) $
babbz:collisions-de-hachage/ (master) $ md5sum original.bmp && md5sum attaque.bmp
de7ee0a85777713e58370d38cd1835cf  original.bmp
bf6ed286e9b80a64f0117602c63835cf  attaque.bmp
```

Figure 7: Déroulement Floyd 3 naif exemple 1

Comme on peut le voir sur le screen,les temps d'executions du programme sont très variables et peuvent aller jusqu'à *131* secondes si aucune collision n'est trouvée. Le temps étant liées au nombre d'itérations réalisés, elles aussi varient énormément en fonction de la seed. Dans la dernière execution, la seed est valide, donc le hash de nos deux images *original.bmp* et *attaque.bmp* collisionnent bien sur les 15 bits demandés `35cf` et cela en *2622* itérations soit *9.45* secondes. Ce cas est particulièrement intéressant car on peut voir que la collision s'effectue sur non pas 15 bits mais sur 21 bits ! Chose que nous n'avons pas réussi a reproduire en spécifiant un *t* de 21 à l'algorithme.

```
    |-------- 21 bits
  1|   8  35cf
110|1 1011 ...
  3|   8  35cf
101|1 1011 ...
```

Optimisations

À ce stade, notre programme est utilisable, mais encore naïf. Les processus sont longs et peu voir non optimisés. Les fichiers `faussaire.c` et `brent.c` présentent un programme `faussaire`, qui vont optimiser le code de plusieurs manières.

D'abord, on inclut la macro `_hash_handler`, qui sans être réellement une optimisation, permet d'alléger le programme. L'utilisation d'une macro permet aussi d'indiquer seulement les paramètres `IMAGE` et `POSITION`, puisque la suite de fonctions invoquée dans une itération de l'algorithme ne dépend que de l'image (entrée ou sortie) et du pointeur de position qu'on suivra (lièvre ou tortue).

```
#define _hash_handler(HASH, IMAGE, POSITION)  \
    free_hfile(POSITION);  \
    f(b_taille, (HASH), (IMAGE), &(POSITION));  \
    md5_small((HASH), b_taille, taille, (POSITION)->data, (POSITION)->
        len);  \
    c.iter++;
```

On choisit aussi une gestion de la mémoire en pile. Cette pile sera libérée à la fin de chaque boucle, permettant d'éviter au programme de prendre une quantité infinie de mémoire (concrètement, la quantité maximale de mémoire prise dépend de la longueur d'une boucle, donc $O(n)$).

Après l'initialisation des différentes variables nécessaires au fonctionnement de l'algorithme, on effectue la première itération de l'algorithme sur Tortue et Lièvre en utilisant l'astuce suivante qui nous permet d'effectuer un calcul de hash en moins: `H(e)=> Tortue` puis `H(Tortue)=> Lièvre`.

L'algorithme de Floyd à été remplacé par celui de Brent comme il suit dans `brent.c`.

```
// Dans collision brent(size_t (*f)(...), h_file, h_file, size_t, hash)
linked array_hash[0x100] = { NULL };
linked pile_mem_original = NULL;
linked pile_mem_attaque = NULL;
linked lookup = NULL;

power = lam = 1;
while(memcmp(h_tortue, h_lievre, b_taille)) {
    if (power == lam) {
        memcpy(h_tortue, h_lievre, b_taille);
        power *= 2;
```

```
        lam = 0;
    }
    _hash_handler(original, lievre);
    empile(lievre, array_hash + h_lievre[b_taille - 1]);
    lam++;
}
```

L'algorithme de Brent est basé sur quatre variables. On garde, comme dans l'algorithme de Floyd, deux pointeurs `tortue` et `lievre`, mais on rajoute les valeurs `power` et `lam`, initialisées à 1. Le lièvre va traverser le graphe nœud par nœud, avec une seule itération par cycle (à la différence de Floyd). On incrémente aussi le compteur `lam` à chaque itération. Lorsque `lam` sera égal à `power`, on remet `lam` à 0, et on copie `h_lievre` dans `h_tortue`.

En essence, le cycle de hash sera de taille n. Lorsque `power` aura dépassé n, le `lievre` traversera forcément la totalité du cycle avant que `lam` dépasse `power`. On atteindra donc la condition de sortie, et `lam` représentera la longueur du cycle.

Notre dernière optimisation à été d'implémenter une table de lookup: `array_hash`. La variable est un tableau de 256 éléments, dans lequel chaque élément est une liste chaînée. Durant le processus on va empiler les images dont on a calculé le hash au bout de la liste chaînée indexée par le dernier byte du hash obtenu. On aura alors un tableau dans lequel tous les hashs pré-calculés de l'image originale sont répartis relativement équitablement dans tout le tableau.

De cette façon, on minimise le temps passé à parcourir des listes chaînées (longueur d'un cycle/256) et la mémoire consommée par cette structure de données (256 pointeurs dans le tas + la mémoire déjà consommée dans la pile par les images). Lors du calcul du hash d'une image de sortie, on va pouvoir rechercher dans la liste chaînée indexée par son dernier octet pour vérifier que ce hash n'est pas déjà calculé. De cette manière, on brise la méthode cyclique pour vérifier qu'un hash n'est pas déjà pré-calculé, mais on observe de gros gains dans le nombre d'itérations. Voici le code associé qui réalise cette optimisation:

```
linked table_lookup(linked *array_hash, hash* h, size_t t, unsigned int
    b_t) {
    hash index = h[b_t - 1];
    linked p = array_hash[index];

    if(p == NULL)
        return 0;

    hash *res = (hash*) calloc(b_t, sizeof(hash));
    while(p != NULL) {
        md5_small(res, b_t, t, p->rec->data, p->rec->len);
        if(!memcmp(res, h, b_t))
            break;
        p = p->nxt;
    }
```

```
    free(res);
    return p;
}
```

La fonction est appelé lors de la troisième phase de l'algorithme pour qu'a chaque itération une recherche dans la table comme expliqué précédemment soit réalisé.

Déroulement

L'algorithme final et optimisé s'appel de la façon suivante:

```
Usage: ./faussaire <entree.bmp> <sortie.bmp> <original.bmp> <attaque.bmp> <taille>
```

L'exemple sur *15* bits permet de nous rendre compte de l'amélioration sur l'optimisation de l'algorithme:

```
babbz:collisions-de-hachage/ (master) $ time ./faussaire tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15
Itérations: 519
./faussaire tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15  2.11s user 0.10s system 99% cpu 2.214 total
babbz:collisions-de-hachage/ (master) $ time ./faussaire tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15
Itérations: 464
./faussaire tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15  2.04s user 0.06s system 99% cpu 2.109 total
babbz:collisions-de-hachage/ (master) $ time ./faussaire tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15
Itérations: 542
./faussaire tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15  2.24s user 0.10s system 99% cpu 2.343 total
babbz:collisions-de-hachage/ (master) $ time ./faussaire tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15
Itérations: 607
./faussaire tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15  2.75s user 0.11s system 99% cpu 2.869 total
babbz:collisions-de-hachage/ (master) $ time ./faussaire tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15
Itérations: 1694
./faussaire tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 15  10.24s user 0.12s system 99% cpu 10.387 total
babbz:collisions-de-hachage/ (master) $ md5sum original.bmp && md5sum attaque.bmp
556bffdfdcb70b2dc727498c6308719f  original.bmp
27b675a5476dfea19a73c89594acf19f  attaque.bmp
```

Figure 8: Déroulement Faussaire exemple 1

En comparaison avec le Floyd_3 naif, dont les résultats furent très aléatoire et le pourcentage de réussite assez faible, le programme *faussaire* lui est bien plus constant. L'algorithme naif qui pouvait mettre jusqu'à 2 minutes, l'algorithme optimisé lui déroule les mêmes résultats en moins de *3* secondes en moyenne. Soit environ *65* fois plus rapidement. Le nombre d'itération est aussi drastiquement plus petit, environ *500* pour l'algorithme optimisé et plus que *10000* pour le naif…

Nos recherches nous ont permis de montrer une collision sur *23* bits au maximum comme le montre le screen ci-dessous. Cette collision à été trouvée en *11238* itérations et *317.38* secondes:

```
babbz:collisions-de-hachage/ (master) $ time ./faussaire tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 23
Itérations: 11238
./faussaire tests/clown.bmp tests/lena.bmp original.bmp attaque.bmp 23  317.38s user 1.53s system 99% cpu 5:19.40 total
babbz:collisions-de-hachage/ (master) $ md5sum original.bmp && md5sum attaque.bmp
c5215fd1dab20ad6b80dd1963808b402  original.bmp
35879bdd350e3fc27c1104a94a88b402  attaque.bmp
```

Figure 9: Déroulement Faussaire exemple 2

Pour finir, voici un tableau qui montre l'évolution du temps d'execution en fonction du nombre de bits en collision sur l'algorithme de Brent:

# bits	Moyenne [ms]	Min [ms]	Max [ms]
10	356.1 ± 75.3	233.6	430.2
11	407.6 ± 120.0	311.7	611.5
12	705.1 ± 208.2	565.9	1069.4
13	1601.8 ± 1256.3	487.4	3000.4
14	6414.2 ± 3888.8	2280.4	11943.1
15	2616.6 ± 925.4	1534.4	3487.2
16	7188.0 ± 1302.5	5610.2	8613.6
17	39457.5 ± 52579.9	7393.7	131615.0
18	18863.6 ± 15233.3	8675.1	45838.5
19	37060.3 ± 18960.0	22233.8	69066.4
20	37650.8 ± 27163.6	14822.3	82585.6

Comme dit précédemment, les temps de calculs vont grandement varier en fonction de la seed initiale. Les calculs étant assez longs à réaliser, chaque lancement du benchmark sur t bits à été lancé seulement 5 fois. Ce qui est trop peu pour avoir des résultats précis mais suffisant pour se faire une idée.